

Alien Invasion — Project Notes

Dự án: Alien Invasion (Python Crash Course)

1. Đúc kết chung từ toàn bộ dự án

- Phát triển thực tế: viết code đơn giản nhất có thể trước, sau đó refactor dần khi dự án phức tạp lên — không cần hoàn hảo ngay từ đầu. (Ví dụ: ship từ trái/phải → 4 hướng; từ 1 file game_functions phình to → tách nhỏ start_game() dùng chung cho cả nút Play và phím P.)
- Đọc hiểu 1 class nhanh: nhìn vào `__init__()` để biết class đó cần input gì (tham số constructor) và tạo ra thuộc tính gì — cách nhanh nhất để nắm class làm được gì mà không cần đọc hết toàn bộ code.
- Bất kỳ chỗ nào set lại rect bằng tay (vd `center_ship()`), phải nhớ đồng bộ lại `self.x/self.y` theo — nếu không, frame kế tiếp code cộng dồn (`self.rect.x = self.x`) sẽ ghi đè mất thay đổi vừa làm.
- Tốc độ game phải độc lập với tốc độ máy: luôn dùng `clock.tick(FPS)` trong vòng lặp chính, nếu không tốc độ di chuyển của mọi vật thể sẽ phụ thuộc vào CPU đang rảnh hay bận, gây cảm giác game nhanh chậm thất thường.

2. Class & Object — vai trò cơ bản

- Class = khuôn mẫu định nghĩa thuộc tính (attributes) và hành vi (methods) mặc định.
- Object = một thực thể cụ thể được tạo ra từ class, dùng để truy cập/thao tác các thuộc tính & hàm đó.
- Object thường được truyền làm tham số cho hàm/class khác cần dùng đến nó, hoặc gán thành biến độc lập dùng trực tiếp trong vòng lặp chính (vd: ship trong `while True:`).

Ví dụ: Settings là class lưu các giá trị cấu hình mặc định (kích thước màn hình, màu nền...). Nó đóng vai trò như kho lưu trữ thuộc tính dùng chung, để các class/hàm khác (Ship, Alien, game_functions...) lấy ra dùng mà không cần hard-code số liệu rải rác khắp nơi.

Thuộc tính vừa có giá trị mặc định, vừa cần đổi được linh hoạt

Khi 1 thuộc tính cần giá trị mặc định lúc khởi tạo, nhưng cũng cần reset lại nhiều lần trong lúc chơi (không chỉ 1 lần duy nhất ở `__init__`), tách nó ra thành 1 hàm riêng, và gọi hàm đó trong `__init__`:

```
class Settings:
    def __init__(self):
        # Giá trị cơ bản, không đổi trong suốt ván chơi
        self.screen_width = 1200
        self.screen_height = 600
        self.speedup_scale = 1.1

        self.initialize_dynamic_settings() # set các giá trị có thể đổi

    def initialize_dynamic_settings(self):
        """Các giá trị sẽ bị reset mỗi khi bắt đầu ván mới."""
        self.ship_speed_factor = 1.5
        self.alien_speed_factor = 2
        self.fleet_direction = 1

    def increase_speed(self):
        """Tăng dần do khó theo thời gian chơi."""
        self.ship_speed_factor *= self.speedup_scale
        self.alien_speed_factor *= self.speedup_scale
```

Lợi ích: mỗi khi người chơi bấm Play lại (start_game()), chỉ cần gọi lại initialize_dynamic_settings() để reset về tốc độ ban đầu, mà không đụng đến các giá trị cố định như screen_width.

Áp dụng tương tự cho GameStats — thuộc tính nào không nên reset (vd high_score) thì đặt ngoài reset_stats(), chỉ set 1 lần duy nhất trong __init__:

```
class GameStats:
    def __init__(self, ai_settings):
        self.ai_settings = ai_settings
        self.reset_stats()

        self.high_score = 0      # không dat trong reset_stats()
        self.game_active = False # -> không bị mất khi chơi lại

    def reset_stats(self):
        """Cac gia tri se ve lai mac dinh moi khi bat dau van moi."""
        self.ship_left = self.ai_settings.ship_limit
        self.score = 0
        self.level = 1
```

3. Pygame — vai trò các hàm hay dùng

Hàm	Vai trò
pygame.init()	Khởi tạo các thiết lập nền cần thiết để Pygame hoạt động
pygame.display.set_mode((w, h))	Tạo cửa sổ game, trả về 1 Surface đại diện toàn bộ màn hình
pygame.display.set_caption()	Đặt tiêu đề cửa sổ
pygame.event.get()	Lấy danh sách sự kiện vừa xảy ra (phím, chuột, đóng cửa sổ...)
pygame.display.flip()	Vẽ lại toàn bộ màn hình với vị trí mới, ẩn vị trí cũ
pygame.image.load()	Load ảnh từ file, trả về 1 Surface đại diện ảnh đó
pygame.draw.rect(screen, color, rect)	Vẽ 1 hình chữ nhật lên màn hình theo màu & rect
pygame.time.Clock() + clock.tick(FPS)	Giới hạn số khung hình/giây, giữ tốc độ game ổn định trên mọi máy

Lưu ý tọa độ: gốc (0, 0) nằm ở góc trên-trái màn hình, trục x tăng dần sang phải, trục y tăng dần xuống dưới (ngược với hình dung toán học thông thường).

Sprite — nhóm các object cùng loại

- Sprite: dùng cho các đối tượng có tính chất 'nhân bản' — nhiều thực thể cùng loại xuất hiện đồng thời (vd: nhiều alien, nhiều bullet). Nếu 1 loại object có thể có nhiều bản sao cùng lúc, biến nó thành Sprite để dễ quản lý theo nhóm.
- pygame.sprite.Group: hoạt động như 1 list, có thêm tiện ích cho game — vd gọi group.update() sẽ tự động gọi update() của từng sprite bên trong.
- pygame.sprite.groupcollide(group1, group2, dokill1, dokill2): so sánh rect giữa 2 nhóm, trả về dictionary các cặp va chạm — key là sprite ở group1, value là list sprite ở group2 bị va chạm. Ví dụ: groupcollide(bullets, aliens, False, True) → key là bullet, value là các alien bị bắn trúng.

- `pygame.sprite.spritecollideany(sprite, group)`: kiểm tra 1 sprite đơn có va chạm bất kỳ sprite nào trong group không → trả về sprite đầu tiên va chạm, hoặc None nếu không có.

Rect — thuộc tính & cách dùng

Nhóm	Thuộc tính	Dùng để
Cạnh/mép	top, bottom, left, right	So sánh, kiểm tra vị trí so với biên màn hình/vật khác
Điểm neo	x, y (góc trên-trái), centerx, centery	Gán vị trí mới cho rect

Khi kiểm tra biên màn hình:

```
if self.rect.right < self.screen_rect.right:    # chưa chạm biên phải
if self.rect.left > 0:                          # chưa chạm biên trái
```

Lưu ý quan trọng — rect chỉ lưu số nguyên (int): các thuộc tính của rect (centerx, x, y...) chỉ nhận giá trị int, pygame tự làm tròn. Nếu tốc độ di chuyển là số thập phân nhỏ (vd 0.5), cộng trực tiếp vào rect sẽ mất phần lẻ, khiến vật thể đứng yên không nhúc nhích.

Giải pháp: lưu vị trí 'thật' ở 1 biến float riêng (self.x, self.y), tính cộng trừ trên biến float này, rồi đồng bộ lại vào rect để vẽ:

```
self.y -= self.speed_factor    # tính trên biến float – giữ chính xác phần thập phân
self.rect.y = self.y          # đồng bộ vào rect để vẽ (lúc này mới làm tròn)
```

blit() — vẽ ảnh lên màn hình

```
self.screen.blit(self.image, self.rect)
```

Vẽ self.image lên self.screen, tại vị trí xác định bởi self.rect. Chỉ vẽ 1 lần tại thời điểm gọi — phải gọi lại mỗi frame trong vòng lặp chính để hình ảnh cập nhật liên tục.

4. Ship — di chuyển & khởi tạo

```
class Ship:
    def __init__(self, ai_settings, screen):
        self.screen = screen
        self.ai_settings = ai_settings
        self.image = pygame.image.load("images/ship.bmp")
        self.rect = self.image.get_rect()
        self.screen_rect = self.screen.get_rect()

        # Canh ship ở giữa-dưới màn hình
        self.rect.centerx = self.screen_rect.centerx
        self.rect.bottom = self.screen_rect.bottom

        # Lưu vị trí thật dạng float
        self.x = float(self.rect.x)
        self.y = float(self.rect.y)

        self.moving_right = False
        self.moving_left = False
        self.moving_up = False
        self.moving_down = False
```

```
def update(self):
    if self.moving_right and self.rect.right < self.screen_rect.right:
        self.x += self.ai_settings.ship_speed_factor
    if self.moving_left and self.rect.left > 0:
        self.x -= self.ai_settings.ship_speed_factor
    if self.moving_up and self.rect.top > 0:
        self.y -= self.ai_settings.ship_speed_factor
    if self.moving_down and self.rect.bottom < self.screen_rect.bottom:
        self.y += self.ai_settings.ship_speed_factor
    self.rect.x = self.x
    self.rect.y = self.y
```

4 hướng di chuyển đối xứng nhau theo cặp trục x/y: trái/phải dùng left/right + x, lên/xuống dùng top/bottom + y. Vì trục y tăng dần xuống dưới, nên 'lên' = trừ y, 'xuống' = cộng y.

center_ship() — reset vị trí ship sau khi mất mạng

Lỗi thường gặp: chỉ gán self.center (không tồn tại, không có tác dụng gì) mà quên set lại rect.centerx/rect.bottom, và quên đồng bộ self.x/self.y — khiến update() ở frame kế ghi đè lại vị trí cũ.

Cách viết đúng — lặp lại đúng logic đã có trong __init__():

```
def center_ship(self):
    """Center the ship on the screen."""
    self.rect.centerx = self.screen_rect.centerx
    self.rect.bottom = self.screen_rect.bottom

    # Đồng bộ lại vị trí thật, để update() không ghi đè vị trí cũ
    self.x = float(self.rect.x)
    self.y = float(self.rect.y)
```

5. Bullet — bắn đạn

```
class Bullet(Sprite):
    def __init__(self, ai_settings, screen, ship):
        super().__init__()
        self.screen = screen
        self.rect = pygame.Rect(0, 0, ai_settings.bullet_width,
ai_settings.bullet_height)
        self.rect.centerx = ship.rect.centerx
        self.rect.top = ship.rect.top
        self.y = float(self.rect.y)
        self.color = ai_settings.bullet_color
        self.speed_factor = ai_settings.bullet_speed_factor

    def update(self):
        self.y -= self.speed_factor
        self.rect.y = self.y
```

Bullet kế thừa Sprite để có thể gom vào Group() — nhờ vậy chỉ cần gọi bullets.update() 1 lần là tất cả viên đạn tự cập nhật vị trí. Đạn được tạo tại đúng vị trí ship (centerx + top) rồi tự bay lên (y giảm dần).

6. Khởi động & reset game

game_active mặc định phải là False

Nếu GameStats khởi tạo `game_active = True`, game sẽ chạy ngay khi mở lên, nút Play không có cơ hội hiện ra (vì điều kiện 'not stats.game_active' để bấm Play luôn sai). Mặc định đúng là False, chỉ chuyển True khi người chơi bấm Play hoặc bấm phím tắt.

start_game() — hàm dùng chung cho nút Play và phím tắt

Tách phần logic 'bắt đầu game' (reset stats, tạo fleet, center ship, ẩn chuột...) ra 1 hàm riêng, để cả `check_play_button()` (bấm chuột) lẫn `check_keydown_events()` (bấm phím P) đều gọi chung, tránh lặp code:

```
def start_game(ai_settings, screen, stats, ship, aliens, bullets):
    """Start a new game."""
    pygame.mouse.set_visible(False)
    stats.reset_stats()
    stats.game_active = True
    aliens.empty()
    bullets.empty()
    create_fleet(ai_settings, screen, ship, aliens)
    ship.center_ship()
```

Trong `check_keydown_events()`, nhớ chỉ cho phép bấm P khi game chưa active, tránh reset nhầm giữa lúc đang chơi:

```
elif event.key == pygame.K_p:
    if not stats.game_active:
        start_game(ai_settings, screen, stats, ship, aliens, bullets)
```

clock.tick(FPS) — chống tốc độ game phụ thuộc CPU

Thiếu giới hạn khung hình khiến vòng lặp game chạy nhanh hết mức máy cho phép — tốc độ mọi vật thể (ship, alien, bullet) phụ thuộc vào CPU đang rảnh hay bận tại từng thời điểm, gây cảm giác lúc nhanh lúc chậm thất thường. Thêm Clock để cố định FPS:

```
clock = pygame.time.Clock()

while True:
    gf.check_events(...)
    if stats.game_active:
        ship.update()
        gf.update_bullets(...)
        gf.update_aliens(...)
    gf.update_screen(...)
    clock.tick(60) # giới hạn 60 khung hình/giay
```

Scoreboard — thu nhỏ icon mạng sống riêng biệt với ship thật

Icon mạng sống trong `prep_ship()` dùng chung class Ship với ship thật trong game nên bị to. Scale nhỏ thêm 1 lần nữa cho riêng icon này, không ảnh hưởng ship đang điều khiển (vì đây là object Ship mới, tách biệt):

```
def prep_ship(self):
    """Show how many ships are left."""
    self.ships = Group()
    for ship_number in range(self.stats.ship_left):
```

```
ship = Ship(self.ai_settings, self.screen)

life_icon_scale = 0.5
new_width = int(ship.rect.width * life_icon_scale)
new_height = int(ship.rect.height * life_icon_scale)
ship.image = pygame.transform.smoothscale(ship.image, (new_width, new_height))
ship.rect = ship.image.get_rect()

ship.rect.x = 10 + ship_number * ship.rect.width
ship.rect.y = 10
self.ships.add(ship)
```

Alien Invasion — Project Notes

Project: Alien Invasion (Python Crash Course)

1. Overall takeaways from the project

- Realistic development process: write the simplest possible code first, then refactor as the project grows more complex — it doesn't need to be perfect from the start. (Example: ship movement went from left/right only to 4 directions; a single bloated `game_functions` file was later split, extracting `start_game()` to be shared by both the Play button and the P key.)
- Quick way to understand a class: look at `__init__()` to see what input it needs (constructor parameters) and what attributes it creates — the fastest way to grasp what a class does without reading all the code.
- Any place that manually sets `rect` (e.g. `center_ship()`) must also re-sync `self.x/self.y` — otherwise, on the next frame the accumulation code (`self.rect.x = self.x`) will overwrite the change just made.
- Game speed must be independent of machine speed: always use `clock.tick(FPS)` in the main loop. Without it, every object's movement speed depends on how busy the CPU is at that moment, causing the game to feel inconsistently fast or slow.

2. Class & Object — basic roles

- Class = a blueprint defining default attributes and behaviors (methods).
- Object = a concrete instance created from a class, used to access those attributes and methods.
- Objects are typically passed as parameters to other functions/classes that need them, or kept as standalone variables used directly in the main loop (e.g. ship inside `while True:`).

Example: `Settings` is a class that stores default configuration values (screen size, background color, etc). It acts as a shared storage of attributes, so other classes/functions (`Ship`, `Alien`, `game_functions...`) can pull values from one place instead of hard-coding numbers scattered everywhere.

Attributes that need both a default value and the ability to change

When an attribute needs a default value at initialization, but also needs to be reset multiple times during gameplay (not just once in `__init__`), extract it into its own method and call that method inside `__init__`:

```
class Settings:
    def __init__(self):
        # Fixed values, unchanged throughout the game session
        self.screen_width = 1200
        self.screen_height = 600
        self.speedup_scale = 1.1

        self.initialize_dynamic_settings() # set values that can change

    def initialize_dynamic_settings(self):
        """Values that get reset each time a new game starts."""
        self.ship_speed_factor = 1.5
        self.alien_speed_factor = 2
        self.fleet_direction = 1

    def increase_speed(self):
        """Gradually increase difficulty as the game progresses."""
        self.ship_speed_factor *= self.speedup_scale
        self.alien_speed_factor *= self.speedup_scale
```

Benefit: every time the player clicks Play again (`start_game()`), simply call `initialize_dynamic_settings()` again to reset back to the starting speed, without touching fixed values like `screen_width`.

The same pattern applies to `GameStats` — an attribute that should never be reset (e.g. `high_score`) is placed outside `reset_stats()`, set only once in `__init__`:

```
class GameStats:
    def __init__(self, ai_settings):
        self.ai_settings = ai_settings
        self.reset_stats()

        self.high_score = 0      # not inside reset_stats()
        self.game_active = False # -> survives across new games

    def reset_stats(self):
        """Values that go back to default each time a new game starts."""
        self.ship_left = self.ai_settings.ship_limit
        self.score = 0
        self.level = 1
```

3. Pygame — commonly used functions

Function	Role
<code>pygame.init()</code>	Initializes the background settings Pygame needs to work
<code>pygame.display.set_mode((w, h))</code>	Creates the game window, returns a Surface representing the whole screen
<code>pygame.display.set_caption()</code>	Sets the window title
<code>pygame.event.get()</code>	Retrieves the list of events that just occurred (keypress, mouse, window close...)
<code>pygame.display.flip()</code>	Redraws the whole screen with new positions, hiding the old ones
<code>pygame.image.load()</code>	Loads an image from file, returns a Surface representing that image
<code>pygame.draw.rect(screen, color, rect)</code>	Draws a rectangle onto the screen at the given color and rect
<code>pygame.time.Clock() + clock.tick(FPS)</code>	Caps the frame rate, keeping game speed consistent across machines

Coordinate note: the origin (0, 0) is at the top-left corner of the screen. The x-axis increases to the right, and the y-axis increases downward (the opposite of typical math conventions).

Sprite — grouping objects of the same kind

- **Sprite**: used for objects that can exist as multiple simultaneous instances (e.g. many aliens, many bullets). If a type of object can have multiple copies on screen at once, turn it into a Sprite for easier group management.
- `pygame.sprite.Group`: behaves like a list, with extra game-related conveniences — e.g. calling `group.update()` automatically calls `update()` on every sprite inside it.

- `pygame.sprite.groupcollide(group1, group2, dokill1, dokill2)`: compares rects between two groups, returns a dictionary of colliding pairs — the key is a sprite from group1, the value is a list of sprites from group2 it collided with. Example: `groupcollide(bullets, aliens, False, True)` → key is the bullet, value is the aliens it hit.
- `pygame.sprite.spritecollideany(sprite, group)`: checks whether a single sprite collides with any sprite in a group → returns the first colliding sprite, or None if there's no collision.

Rect — attributes and usage

Group	Attributes	Used for
Edges	top, bottom, left, right	Comparing / checking position relative to the screen edge or another object
Anchor points	x, y (top-left corner), centerx, centery	Assigning a new position to the rect

When checking the screen edge:

```
if self.rect.right < self.screen_rect.right:    # hasn't hit the right edge yet
if self.rect.left > 0:                          # hasn't hit the left edge yet
```

Important note — rect only stores integers: rect attributes (centerx, x, y...) only accept int values; pygame rounds them automatically. If the movement speed is a small decimal (e.g. 0.5), adding it directly to the rect loses the fractional part, causing the object to appear stuck / not moving.

Solution: keep the 'real' position in a separate float variable (`self.x`, `self.y`), do the math on that float, then sync it back into the rect for drawing:

```
self.y -= self.speed_factor    # calculated on the float variable - keeps decimal
precision
self.rect.y = self.y           # synced into rect for drawing (rounding happens here)
```

blit() — drawing an image to the screen

```
self.screen.blit(self.image, self.rect)
```

Draws `self.image` onto `self.screen`, at the position defined by `self.rect`. It only draws once at the moment it's called — it must be called again every frame in the main loop for the image to keep updating.

4. Ship — movement & initialization

```
class Ship:
    def __init__(self, ai_settings, screen):
        self.screen = screen
        self.ai_settings = ai_settings
        self.image = pygame.image.load("images/ship.bmp")
        self.rect = self.image.get_rect()
        self.screen_rect = self.screen.get_rect()

        # Center the ship at the bottom of the screen
        self.rect.centerx = self.screen_rect.centerx
        self.rect.bottom = self.screen_rect.bottom

        # Store the real position as float
```

```

        self.x = float(self.rect.x)
        self.y = float(self.rect.y)

        self.moving_right = False
        self.moving_left = False
        self.moving_up = False
        self.moving_down = False

    def update(self):
        if self.moving_right and self.rect.right < self.screen_rect.right:
            self.x += self.ai_settings.ship_speed_factor
        if self.moving_left and self.rect.left > 0:
            self.x -= self.ai_settings.ship_speed_factor
        if self.moving_up and self.rect.top > 0:
            self.y -= self.ai_settings.ship_speed_factor
        if self.moving_down and self.rect.bottom < self.screen_rect.bottom:
            self.y += self.ai_settings.ship_speed_factor
        self.rect.x = self.x
        self.rect.y = self.y

```

The 4 movement directions mirror each other across the x/y axis pairs: left/right use left/right + x, up/down use top/bottom + y. Since the y-axis increases downward, 'up' means subtracting y, and 'down' means adding y.

center_ship() — resetting the ship's position after losing a life

Common mistake: assigning self.center (which doesn't exist and has no effect) while forgetting to reset rect.centerx/rect.bottom, and forgetting to sync self.x/self.y — which causes the next frame's update() to overwrite it back to the old position.

Correct version — mirrors the same logic already in __init__():

```

def center_ship(self):
    """Center the ship on the screen."""
    self.rect.centerx = self.screen_rect.centerx
    self.rect.bottom = self.screen_rect.bottom

    # Re-sync the real position so update() doesn't overwrite it
    self.x = float(self.rect.x)
    self.y = float(self.rect.y)

```

5. Bullet — firing

```

class Bullet(Sprite):
    def __init__(self, ai_settings, screen, ship):
        super().__init__()
        self.screen = screen
        self.rect = pygame.Rect(0, 0, ai_settings.bullet_width,
ai_settings.bullet_height)
        self.rect.centerx = ship.rect.centerx
        self.rect.top = ship.rect.top
        self.y = float(self.rect.y)
        self.color = ai_settings.bullet_color
        self.speed_factor = ai_settings.bullet_speed_factor

    def update(self):

```

```
self.y -= self.speed_factor
self.rect.y = self.y
```

Bullet inherits from Sprite so it can be placed in a Group() — calling bullets.update() once automatically updates every bullet's position. Each bullet spawns at the ship's position (centerx + top) and then travels upward (y decreasing).

6. Starting & resetting the game

game_active must default to False

If GameStats initializes game_active = True, the game starts running immediately when opened, and the Play button never gets a chance to appear (since the condition 'not stats.game_active' for clicking Play is always false). The correct default is False, switching to True only when the player clicks Play or presses the shortcut key.

start_game() — a shared function for the Play button and keyboard shortcut

Extract the 'start a new game' logic (reset stats, create fleet, center ship, hide the cursor...) into its own function, so both check_play_button() (mouse click) and check_keydown_events() (P key) call the same place, avoiding duplicated code:

```
def start_game(ai_settings, screen, stats, ship, aliens, bullets):
    """Start a new game."""
    pygame.mouse.set_visible(False)
    stats.reset_stats()
    stats.game_active = True
    aliens.empty()
    bullets.empty()
    create_fleet(ai_settings, screen, ship, aliens)
    ship.center_ship()
```

In check_keydown_events(), remember to only allow the P key when the game isn't already active, to avoid accidentally resetting mid-game:

```
elif event.key == pygame.K_p:
    if not stats.game_active:
        start_game(ai_settings, screen, stats, ship, aliens, bullets)
```

clock.tick(FPS) — preventing game speed from depending on the CPU

Without a frame-rate cap, the main loop runs as fast as the machine allows — every object's speed (ship, alien, bullet) depends on how busy the CPU is at any given moment, causing the game to feel inconsistently fast or slow. Add a Clock to fix the FPS:

```
clock = pygame.time.Clock()

while True:
    gf.check_events(...)
    if stats.game_active:
        ship.update()
        gf.update_bullets(...)
        gf.update_aliens(...)
    gf.update_screen(...)
    clock.tick(60)  # cap at 60 frames per second
```

Scoreboard — scaling the life icons separately from the real ship

The life icons in `prep_ship()` reuse the same `Ship` class as the real ship, so they end up too large. Scale them down again just for these icons, without affecting the ship currently being controlled (since this is a new, separate `Ship` object):

```
def prep_ship(self):
    """Show how many ships are left."""
    self.ships = Group()
    for ship_number in range(self.stats.ship_left):
        ship = Ship(self.ai_settings, self.screen)

        life_icon_scale = 0.5
        new_width = int(ship.rect.width * life_icon_scale)
        new_height = int(ship.rect.height * life_icon_scale)
        ship.image = pygame.transform.smoothscale(ship.image, (new_width, new_height))
        ship.rect = ship.image.get_rect()

        ship.rect.x = 10 + ship_number * ship.rect.width
        ship.rect.y = 10
        self.ships.add(ship)
```